

Topic 5: Stacks

5.1 Stack ADT

- A Stack is a Last-In First-Out (LIFO) data structure
- Operations: push (add to top), pop (remove from top), peek/top (view top without removing), isEmpty
- Real-world analogies: stack of plates, function call stack, undo history
- All operations are $O(1)$

5.2 Array-Based Stack

- Use a fixed array and an integer top (initially -1)
- push: check overflow ($\text{top} == \text{MAX}-1$), then $\text{data}[++\text{top}] = \text{val}$
- pop: check underflow ($\text{top} == -1$), then return $\text{data}[\text{top--}]$
- Simple but limited to a fixed capacity

```
class ArrayStack {
    int data[100], top = -1;
public:
    void push(int v) {
        if (top == 99) { cout << "Overflow"; return; }
        data[++top] = v;
    }
    int pop() { return (top == -1) ? -1 : data[top--]; }
    int peek() { return data[top]; }
    bool empty() { return top == -1; }
};
```

5.3 Linked-List-Based Stack

- Push: insert new node at front of linked list -- $O(1)$
- Pop: remove front node -- $O(1)$
- No capacity limit -- grows as needed

```

class LinkedStack {
    Node* top = nullptr;
public:
    void push(int v) { Node* n = new Node(v); n->next = top; top = n; }
    int pop() { int v = top->data; Node* t = top; top = top->next; delete t; return v; }
    int peek() { return top->data; }
    bool empty(){ return top == nullptr; }
};

```

5.4 Applications of Stacks

- Balanced parentheses checking: push (; pop and match for) -- if stack empty at end, balanced
- Expression evaluation: postfix (Reverse Polish Notation) evaluation using a stack
- Infix to postfix conversion (Shunting-Yard algorithm)
- Function call stack: each call pushes a frame; return pops it
- Depth-First Search (DFS) -- uses a stack (either explicit or the call stack via recursion)

```

bool isBalanced(string expr) {
    stack<char> s;
    for (char c : expr) {
        if (c=='(' || c=='{' || c=='[') s.push(c);
        else if (c==')' || c=='}' || c==']') {
            if (s.empty()) return false;
            s.pop();
        }
    }
    return s.empty();
}

```